

Imam Abdulrahman Bin Faisal University

College of Computer Science and Information Technology
Department of Computer Engineering

ARTI403 - Image Processing Lab 1 Report

Basics of Programming with Python

Student Name	Mohammed Moayed Alquriniy
Student ID	2240007325
Academic Year	2025 - 2026
GitHub Repository	github.com/itsMo33/ARTI403---Image-Processing

1. Objective

The objective of this lab is to explain how digital images are represented and manipulated in a computer, by loading, displaying, and saving images with OpenCV and PIL, and by working with images as NumPy arrays.

2. Tools

Python 3, Anaconda, Jupyter Notebook, and the libraries OpenCV, Pillow, NumPy, and Matplotlib.

3. Implementation and Results

Task #2: Loading and Visualizing Images

a. Loading/reading an image using OpenCV

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('images/cameraman.tif') # Load the image data into a NumPy array

plt.imshow(img)
plt.show()
```

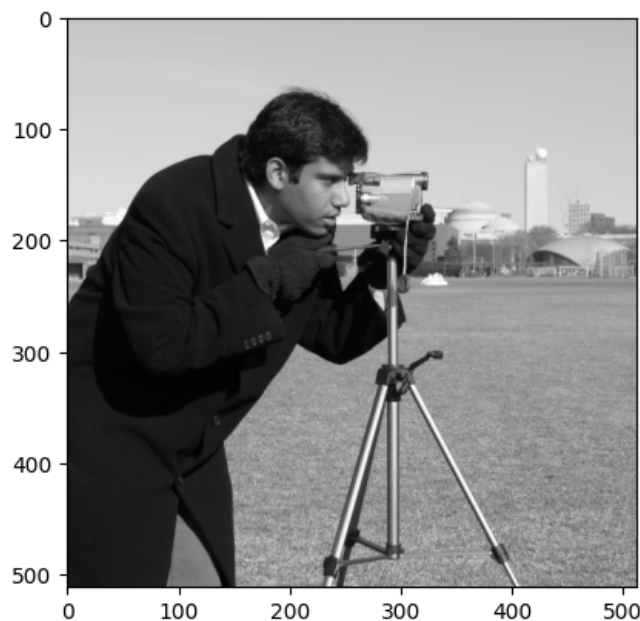


Figure 1

Observation: The image is displayed correctly, but it is shown without converting the color order. Since cameraman.tif is a grayscale image, the three channels B, G and R are equal, so it still looks gray.

Note on colors: OpenCV reads images in **BGR** order, but matplotlib expects **RGB**. Since cameraman.tif is grayscale, OpenCV still loads it as a 3-channel image, so the colors above may look tinted. Convert it for a correct grayscale display:

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(img_rgb, cmap='gray')
plt.title('Cameraman (OpenCV, corrected)')
plt.show()
```

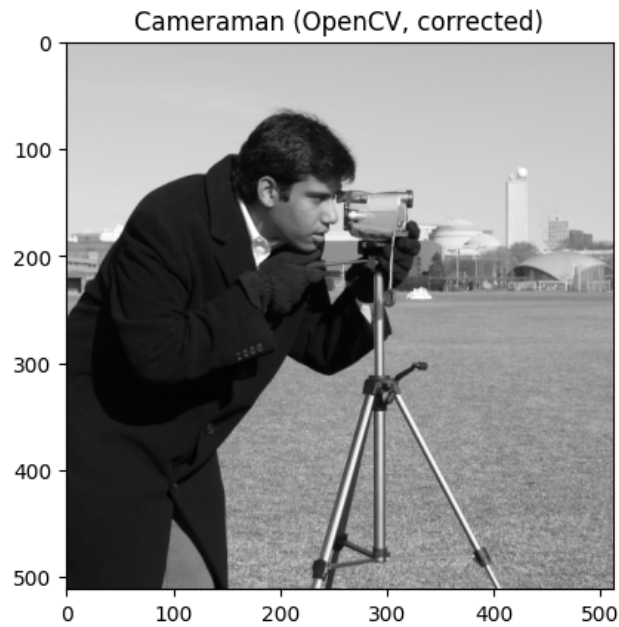


Figure 2

b. Loading/reading image using PIL

```
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.cm as cm

img2 = Image.open('images/lena_gray_256.tif') # load image into a PIL Image object

plt.imshow(img2, cmap=cm.Greys_r)
plt.show()
```

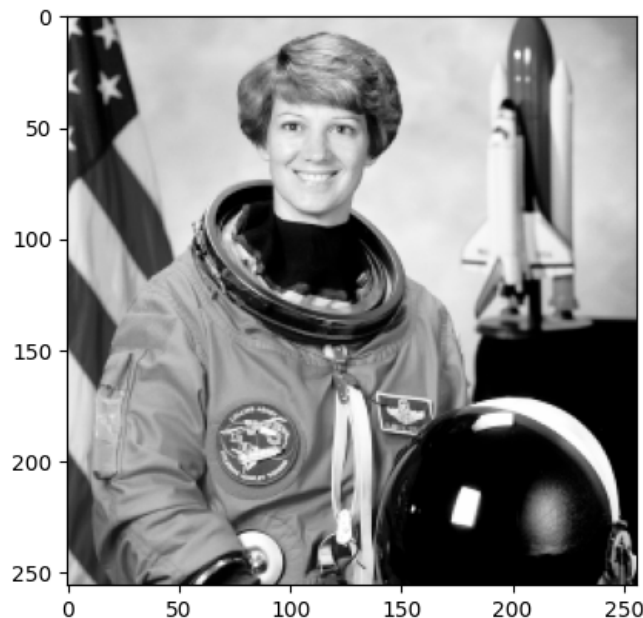


Figure 3

Observation: PIL loads the image as a PIL Image object, and it is displayed in grayscale using the Greys_r colormap.

Task #3: Image Storing

a. Saving image to disk using OpenCV

```
cv2.imwrite('new_image.jpg', img)
```

Output:
True

Observation: The function returned True, which means the image was saved successfully as new_image.jpg.

b. Saving an image to disk using PIL

```
img2.save('new_image2.jpg')
```

Observation: The PIL image was saved as new_image2.jpg in the same folder as the notebook.

Task #4: Display Image as an Array

```
print(img.shape) # Note: img loaded using OpenCV is a NumPy array
print(img)
```

```
Output:
(512, 512, 3)
[[[200 200 200]
  [200 200 200]
  [200 200 200]
  ...
  [189 189 189]
  [190 190 190]
  [190 190 190]]
 [[200 200 200]
  [199 199 199]
  [199 199 199]
  ...
  [190 190 190]
  ...
```

Observation: The image loaded with OpenCV is a NumPy array with shape (512, 512, 3): 512 rows, 512 columns, and 3 channels. All three channels have the same value in each pixel because the image is grayscale.

```
import numpy as np

# Convert the image to a NumPy array
img_array = np.array(img2)

# Print the array
print(img_array.shape)
print(img_array)
```

```
Output:
(256, 256)
[[143  86 113 ... 121 121 118]
 [200 179 185 ... 119 120 118]
 [227 218 213 ... 120 120 119]
 ...
 [174 176 170 ...  62  25   0]
 [176 172 162 ...  47   1   0]
 [172 166 148 ...  84  21   1]]
```

Observation: After converting the PIL image to a NumPy array, the shape is (256, 256) because the image has only one channel. Each value is the intensity of one pixel between 0 and 255.

Assessment

The following tasks were solved using `img_array` to practice NumPy operations on image data:

- Print the minimum, maximum, and mean pixel value.
- Crop a 100×100 region from the top-left corner.
- Flip the image horizontally using array slicing (`img_array[:, ::-1]`).
- Threshold the image: set every pixel above 128 to 255 and every pixel below to 0.

1. Minimum, maximum, and mean pixel value

```
print('Minimum pixel value:', img_array.min())
print('Maximum pixel value:', img_array.max())
print('Mean pixel value:', round(img_array.mean(), 2))
```

```
Output:
Minimum pixel value: 0
Maximum pixel value: 255
Mean pixel value: 115.52
```

The pixel values are between 0 (black) and 255 (white), and the mean shows the average brightness of the image.

2. Crop a 100×100 region from the top-left corner

```
crop = img_array[:100, :100] # rows 0-99 and columns 0-99
print('Cropped shape:', crop.shape)

plt.imshow(crop, cmap='gray', vmin=0, vmax=255)
plt.title('Cropped 100x100 (top-left)')
plt.show()
```

```
Output:
Cropped shape: (100, 100)
```

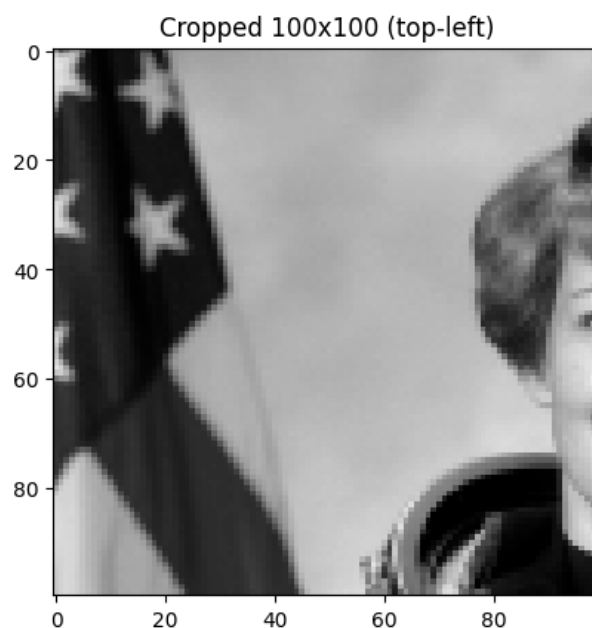


Figure 4

Slicing the first 100 rows and the first 100 columns of the array gives the top-left part of the image.

3. Flip the image horizontally using array slicing

```
flipped = img_array[:, ::-1] # reverse the order of the columns

plt.subplot(1, 2, 1)
plt.imshow(img_array, cmap='gray', vmin=0, vmax=255)
plt.title('Original')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(flipped, cmap='gray', vmin=0, vmax=255)
plt.title('Flipped horizontally')
plt.axis('off')
plt.show()
```

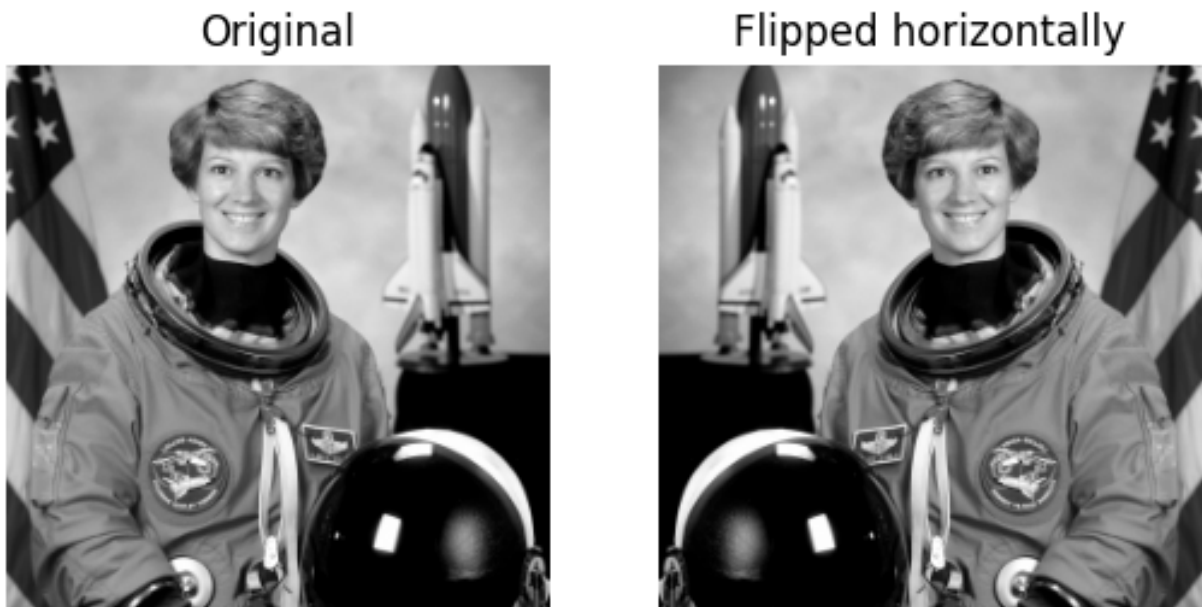


Figure 5

::-1 reverses the columns, so the left side of the image becomes the right side (mirror image). The rows do not change.

4. Threshold the image at 128

```
thresholded = img_array.copy()
thresholded[img_array > 128] = 255
thresholded[img_array <= 128] = 0

plt.subplot(1, 2, 1)
plt.imshow(img_array, cmap='gray', vmin=0, vmax=255)
plt.title('Original')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(thresholded, cmap='gray', vmin=0, vmax=255)
plt.title('Thresholded (128)')
plt.axis('off')
plt.show()

print('Unique values after thresholding:', np.unique(thresholded))
```

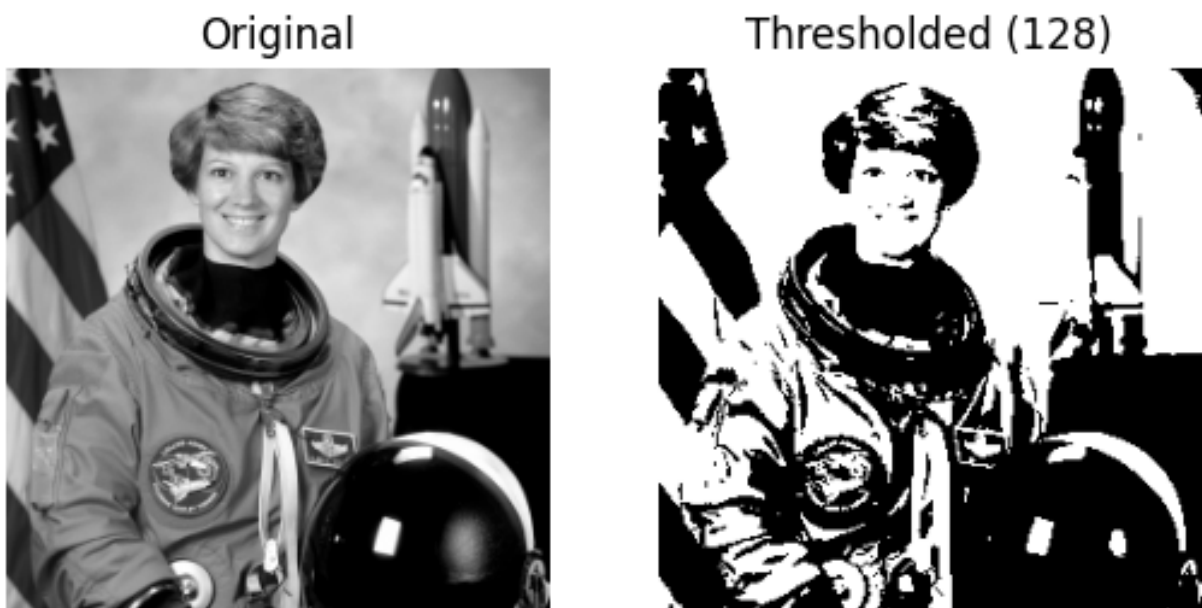


Figure 6

```
Output:
Unique values after thresholding: [  0 255]
```

Every pixel above 128 became white (255) and every other pixel became black (0), so the result is a binary image with only two values.

4. Conclusion

In this lab, images were loaded and displayed using OpenCV and PIL, and saved to disk using both libraries. OpenCV reads images in BGR order, so a conversion is needed before displaying them with Matplotlib. Images were also displayed as NumPy arrays, which showed that a digital image is a matrix of pixel values between 0 and 255. Finally, simple NumPy operations were used to find the minimum, maximum, and mean values, crop the image, flip it horizontally, and threshold it into a binary image.